

阶段五：Baobab-Studio 五阶段 AI 流水线 + 多角色 UI (10 周)

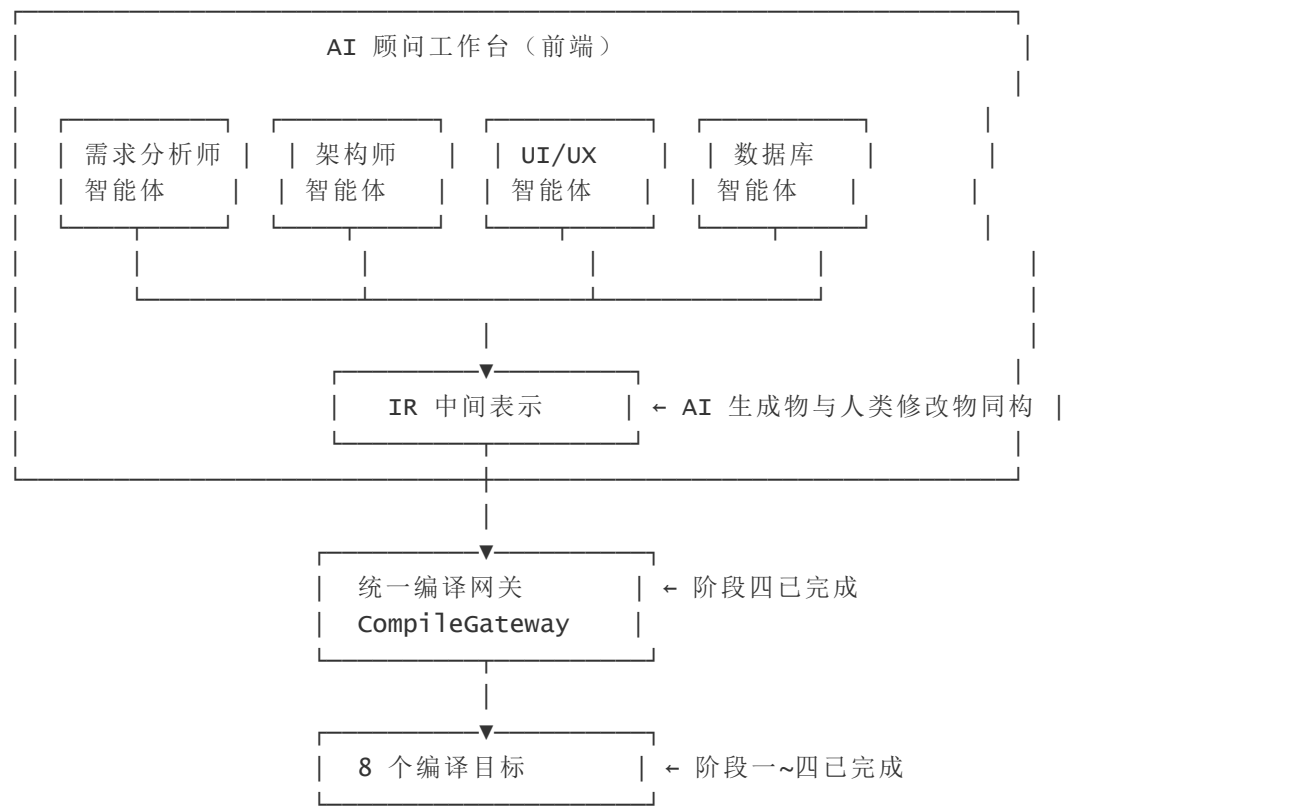
v3.0 对齐 D 爷确定稿第一部分：独立智能体能力全部保留，编排形态升级为五阶段流水线（需求→架构→总体设计→自动开发→交付）；OrchestratorAgent 协调子智能体，每阶段输出 Markdown + IR 契约。工期 10 周不变，不压缩为 8 周。

目标

实现 "顾问式 AI"，让 AI 成为架构师和开发者的决策合伙人。
人类始终在环路中，AI 降智时可无缝降级为专家模式（VisualDev + ir-to-schema 逃生舱）。

- 核心转变：
- AI 不是填表格的 "填表员"
 - AI 是能讨论方案优劣的 "决策合伙人"
 - AI 能主动追问业务潜规则、提供策略选项、分析影响
 - 五阶段流水线对用户呈现单一进度条，对内仍调用 F-9 全部智能体能力

架构总览



Week 1-2: 大模型网关

F-9.1 大模型网关抽象层

```
/**
 * 大模型网关
 *
 * 统一对接多个大模型供应商：
 *   - DeepSeek（国产，性价比高）
 *   - 通义千问（阿里，生态好）
 *   - OpenAI GPT（通用能力最强）
 *   - 本地模型（ollama，离线可用）
 *
 * 核心能力：
 *   1. 统一接口（不同供应商同一套 API）
 *   2. 请求队列（防止并发过载）
 *   3. 失败重试（指数退避）
 *   4. 降智熔断（响应质量过低时自动切换）
 *   5. Token 计量（成本控制）
 *   6. 审计日志（BASE_AI_CALL_LOG 激活）
 */

export interface LLMConfig {
  /** 供应商 */
  provider: 'deepseek' | 'tongyi' | 'openai' | 'ollama';
  /** API Key */
  apiKey?: string;
  /** API Base URL */
  baseUrl: string;
  /** 模型名称 */
  model: string;
  /** 最大 Token 数 */
  maxTokens?: number;
  /** 温度参数（0-1，越低越确定） */
  temperature?: number;
  /** 是否启用流式输出 */
  stream?: boolean;
}

export interface ChatMessage {
  role: 'system' | 'user' | 'assistant';
  content: string;
}

export interface ChatRequest {
  messages: ChatMessage[];
  /** 期望的输出格式 */
  responseFormat?: 'text' | 'json';
  /** 最大重试次数 */
  maxRetries?: number;
  /** 超时时间（ms） */
  timeout?: number;
}

export interface ChatResponse {
  content: string;
  usage: {
```

```

    promptTokens: number;
    completionTokens: number;
    totalTokens: number;
  };
  model: string;
  provider: string;
  latency: number; // ms
}

/**
 * 大模型网关接口
 */
export interface LLMGateway {
  /** 单次对话 */
  chat(request: ChatRequest): Promise<ChatResponse>;
  /** 流式对话 */
  chatStream(request: ChatRequest): AsyncGenerator<string>;
  /** 健康检查 */
  healthCheck(): Promise<boolean>;
  /** 获取当前供应商信息 */
  getProviderInfo(): { provider: string; model: string };
}

/**
 * DeepSeek 实现（推荐首选，性价比最高）
 */

import type { LLMGateway, ChatRequest, ChatResponse, LLMConfig } from './types';

export class DeepSeekGateway implements LLMGateway {
  private config: LLMConfig;
  private requestCount = 0;
  private totalTokens = 0;

  constructor(config: LLMConfig) {
    this.config = config;
  }

  async chat(request: ChatRequest): Promise<ChatResponse> {
    const start = Date.now();
    const maxRetries = request.maxRetries ?? 3;
    let lastError: Error | null = null;

    for (let attempt = 0; attempt < maxRetries; attempt++) {
      try {
        const response = await fetch(`${this.config.baseUrl}/v1/chat/completions`, {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${this.config.apikey}`,
          },
          body: JSON.stringify({
            model: this.config.model,
            messages: request.messages,
            max_tokens: this.config.maxTokens ?? 4096,
            temperature: this.config.temperature ?? 0.7,
            response_format: request.responseFormat === 'json'
              ? { type: 'json_object' }

```

```

        : undefined,
    }},
    signal: AbortSignal.timeout(request.timeout ?? 60000),
});

if (!response.ok) {
    throw new Error(`HTTP ${response.status}: ${response.statusText}`);
}

const data = await response.json() as any;
const content = data.choices?.[0]?.message?.content ?? '';
const usage = data.usage ?? { prompt_tokens: 0, completion_tokens: 0,
total_tokens: 0 };

// 计量
this.requestCount++;
this.totalTokens += usage.total_tokens;

return {
    content,
    usage: {
        promptTokens: usage.prompt_tokens,
        completionTokens: usage.completion_tokens,
        totalTokens: usage.total_tokens,
    },
    model: data.model ?? this.config.model,
    provider: 'deepseek',
    latency: Date.now() - start,
};
} catch (e) {
    lastError = e as Error;
    // 指数退避
    if (attempt < maxRetries - 1) {
        await new Promise(r => setTimeout(r, Math.pow(2, attempt) * 1000));
    }
}
}

throw new Error(`DeepSeek 调用失败（${maxRetries} 次重试后）：
${lastError?.message}`);
}

async *chatStream(request: ChatRequest): AsyncGenerator<string> {
    const response = await fetch(`${this.config.baseUrl}/v1/chat/completions`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${this.config.apiKey}`,
        },
        body: JSON.stringify({
            model: this.config.model,
            messages: request.messages,
            max_tokens: this.config.maxTokens ?? 4096,
            temperature: this.config.temperature ?? 0.7,
            stream: true,
        }),
    });
};

```

```

const reader = response.body?.getReader();
if (!reader) throw new Error('无法获取流式响应');

const decoder = new TextDecoder();
let buffer = '';

while (true) {
  const { done, value } = await reader.read();
  if (done) break;

  buffer += decoder.decode(value, { stream: true });
  const lines = buffer.split('\n');
  buffer = lines.pop() ?? '';

  for (const line of lines) {
    if (line.startsWith('data: ') && line !== 'data: [DONE]') {
      try {
        const data = JSON.parse(line.slice(6));
        const content = data.choices?.[0]?.delta?.content;
        if (content) yield content;
      } catch { /* 忽略解析错误 */ }
    }
  }
}

async healthCheck(): Promise<boolean> {
  try {
    const response = await this.chat({
      messages: [{ role: 'user', content: 'ping' }],
      timeout: 5000,
      maxRetries: 1,
    });
    return response.content.length > 0;
  } catch {
    return false;
  }
}

getProviderInfo() {
  return { provider: 'deepseek', model: this.config.model };
}

/** 获取使用统计 */
getUsageStats() {
  return {
    requestCount: this.requestCount,
    totalTokens: this.totalTokens,
  };
}

/**
 * 多供应商降级网关
 *
 * 主供应商失败时自动切换到备用供应商
 * 实现"降智熔断"--当主供应商响应质量过低时切换
 */

```

```

import type { LLMGateway, ChatRequest, ChatResponse } from './types';

export class FallbackLLMGateway implements LLMGateway {
  private gateways: LLMGateway[];
  private currentIndex = 0;
  private failureCounts: number[];
  private readonly maxFailures = 3;

  constructor(gateways: LLMGateway[]) {
    this.gateways = gateways;
    this.failureCounts = new Array(gateways.length).fill(0);
  }

  async chat(request: ChatRequest): Promise<ChatResponse> {
    let lastError: Error | null = null;

    for (let i = 0; i < this.gateways.length; i++) {
      const idx = (this.currentIndex + i) % this.gateways.length;
      const gateway = this.gateways[idx];

      try {
        const response = await gateway.chat(request);

        // 成功，重置失败计数
        this.failureCounts[idx] = 0;
        this.currentIndex = idx;

        return response;
      } catch (e) {
        lastError = e as Error;
        this.failureCounts[idx]++;

        // 如果连续失败超过阈值，切换到下一个供应商
        if (this.failureCounts[idx] >= this.maxFailures) {
          console.warn(`[LLM] 供应商 ${idx} 连续失败 ${this.maxFailures} 次，切换到备用
供应商`);
          this.currentIndex = (idx + 1) % this.gateways.length;
        }
      }
    }

    throw new Error(`所有 LLM 供应商都失败: ${lastError?.message}`);
  }

  async *chatStream(request: ChatRequest): AsyncGenerator<string> {
    // 流式模式使用当前主供应商
    const gateway = this.gateways[this.currentIndex];
    yield* gateway.chatStream(request);
  }

  async healthCheck(): Promise<boolean> {
    // 检查所有供应商健康状态
    const results = await Promise.allSettled(
      this.gateways.map(g => g.healthCheck())
    );
    return results.some(r => r.status === 'fulfilled' && r.value === true);
  }
}

```

```

    getProviderInfo() {
      return this.gateways[this.currentIndex].getProviderInfo();
    }
  }
}

```

F-9.2 Prompt 模板管理

```

/**
 * Prompt 模板管理
 *
 * 为每个智能体维护 System Prompt 模板
 * 模板中可嵌入 IR 知识、领域知识、EAB 约束等上下文
 */

export interface PromptTemplate {
  id: string;
  name: string;
  /** System Prompt 模板（支持 {{变量}} 插值） */
  systemPrompt: string;
  /** 变量定义 */
  variables: { name: string; description: string; required: boolean }[];
}

/**
 * 需求分析师智能体的 Prompt
 */
export const REQUIREMENT_ANALYST_PROMPT: PromptTemplate = {
  id: 'requirement-analyst',
  name: '需求分析师',
  systemPrompt: `你是一位资深的企业级软件需求分析师，精通 JNPF 低代码平台。

```

你的职责：

1. 理解用户的业务需求（可能是模糊的、不完整的）
2. 主动追问业务潜规则（用户"没说"但"默认"的规则）
3. 将需求转化为结构化的领域模型和用户故事
4. 识别需求中的冲突和矛盾

你的知识背景：

- JNPF 平台支持的领域：{{domains}}
- 已有的领域模式：{{domainPatterns}}
- 平台的技术约束：{{technicalConstraints}}

你的工作方式：

1. 先理解用户的核心诉求（不要急于给方案）
2. 主动追问关键问题（至少 3 个问题）
3. 基于领域知识提供策略选项（不是唯一方案）
4. 分析每个选项的利弊和影响
5. 让用户做最终决策

输出格式（JSON）：

```

{
  "understanding": "对需求的理解",
  "questions": ["追问的问题1", "追问的问题2"],
  "proposedDomainModel": {
    "entities": [...],
    "relationships": [...],
    "businessRules": [...]
  }
}

```

```

    },
    "strategies": [
      {
        "name": "策略名称",
        "description": "策略描述",
        "pros": ["优点1"],
        "cons": ["缺点1"],
        "impact": "影响分析"
      }
    ],
    "userStories": [
      {
        "role": "用户角色",
        "action": "操作",
        "goal": "目标",
        "acceptance": "验收标准"
      }
    ]
  },
  variables: [
    { name: 'domains', description: '平台支持的业务领域列表', required: true },
    { name: 'domainPatterns', description: '已有的领域模式（来自知识图谱）', required: true },
    { name: 'technicalConstraints', description: '平台的技术约束（来自 EAB）', required: true },
  ],
};

```

```

/**
 * 架构师智能体的 Prompt
 */
export const ARCHITECT_PROMPT: PromptTemplate = {
  id: 'architect',
  name: '架构师',
  systemPrompt: `你是一位资深的企业级软件架构师，精通 JNPF 低代码平台的架构设计。

```

你的职责：

1. 基于需求分析师输出的领域模型，设计系统架构
2. 选择技术方案（从 EAB 白名单中选择）
3. 生成架构描述 IR（表单、列表、大屏、API）
4. 确保架构符合 EAB 约束

EAB（企业架构基准）：

```
{{eab}}
```

当前架构约束：

- 部署架构：模块化单体（非微服务）
- 数据库：SQL Server + SqlSugar ORM
- 缓存：Redis（CSRedis）
- 消息队列：RabbitMQ
- 前端：Vue 3 + Ant Design Vue（web）/ wot-design-uni（小程序）

输出格式（JSON）：

```

{
  "architecture": {
    "modules": [...],
    "databaseDesign": {...},
    "apiDesign": {...},

```



```

        "uiDesign": {...}
    },
    "ir": {
        "pages": [...],
        "entities": [...],
        "apis": [...]
    },
    "techStack": {
        "framework": "...",
        "ui": "...",
        "database": "..."
    }
},
variables: [
    { name: 'eab', description: '企业架构基准配置', required: true },
],
};

```

Week 3-4: 需求分析师 + 架构师智能体

F-9.3 智能体基类

```

/**
 * 智能体基类
 *
 * 所有智能体的公共逻辑：
 * 1. 加载 Prompt 模板
 * 2. 填充变量（知识图谱、EAB、IR 等）
 * 3. 调用大模型
 * 4. 解析响应
 * 5. 审计日志
 */

import type { LLMGateway, ChatMessage, ChatResponse } from '../llm/types';
import type { PromptTemplate } from '../llm/prompts';

export interface AgentContext {
    /** 当前对话历史 */
    messages: ChatMessage[];
    /** 知识图谱数据 */
    knowledgeGraph?: Record<string, unknown>;
    /** EAB 配置 */
    eab?: Record<string, unknown>;
    /** 当前 IR（如有） */
    currentIR?: Record<string, unknown>;
    /** 业务领域 */
    domain?: string;
}

export interface AgentResponse<T = unknown> {
    /** 解析后的结构化数据 */
    data: T;
    /** 原始响应文本 */
    rawText: string;
    /** 使用统计 */
    usage: ChatResponse['usage'];
}

```

```

    /** 延迟 (ms) */
    latency: number;
    /** 置信度 (0-1) */
    confidence: number;
}

export abstract class BaseAgent {
    protected llm: LLMGateway;
    protected template: PromptTemplate;

    constructor(llm: LLMGateway, template: PromptTemplate) {
        this.llm = llm;
        this.template = template;
    }

    /**
     * 执行智能体任务
     */
    async execute<T>(userInput: string, context: AgentContext):
    Promise<AgentResponse<T>> {
        // Step 1: 构建系统 Prompt (填充变量)
        const systemPrompt = this.buildSystemPrompt(context);

        // Step 2: 构建消息列表
        const messages: ChatMessage[] = [
            { role: 'system', content: systemPrompt },
            ...context.messages,
            { role: 'user', content: userInput },
        ];

        // Step 3: 调用大模型
        const response = await this.llm.chat({
            messages,
            responseFormat: 'json',
            timeout: 120000, // 智能体可能需要较长时间
        });

        // Step 4: 解析响应
        const data = this.parseResponse<T>(response.content);

        // Step 5: 计算置信度
        const confidence = this.evaluateConfidence(data, response);

        return {
            data,
            rawText: response.content,
            usage: response.usage,
            latency: response.latency,
            confidence,
        };
    }

    /**
     * 流式执行 (用于实时展示 AI 思考过程)
     */
    async *executeStream(userInput: string, context: AgentContext):
    AsyncGenerator<string> {
        const systemPrompt = this.buildSystemPrompt(context);

```

```

const messages: ChatMessage[] = [
  { role: 'system', content: systemPrompt },
  ...context.messages,
  { role: 'user', content: userInput },
];

yield* this.llm.chatStream({ messages });
}

/**
 * 构建系统 Prompt（填充变量）
 */
protected buildSystemPrompt(context: AgentContext): string {
  let prompt = this.template.systemPrompt;

  for (const variable of this.template.variables) {
    const value = this.resolveVariable(variable.name, context);
    if (value !== undefined) {
      prompt = prompt.replace(`${variable.name}`, JSON.stringify(value, null,
2));
    } else if (variable.required) {
      console.warn(`[Agent] 必需变量 ${variable.name} 未提供`);
    }
  }

  return prompt;
}

/**
 * 解析大模型响应为结构化数据
 */
protected parseResponse<T>(content: string): T {
  try {
    // 尝试直接解析 JSON
    return JSON.parse(content) as T;
  } catch {
    // 尝试提取 JSON 块
    const jsonMatch = content.match(/```json\s*([\s\S]*?)\s*```/);
    if (jsonMatch) {
      return JSON.parse(jsonMatch[1]) as T;
    }
    // 尝试提取花括号内容
    const braceMatch = content.match(/\{([\s\S]*)\}/);
    if (braceMatch) {
      return JSON.parse(braceMatch[0]) as T;
    }
    throw new Error('无法解析大模型响应为 JSON');
  }
}

/**
 * 评估响应置信度
 */
protected evaluateConfidence(data: unknown, response: ChatResponse): number {
  // 基础置信度
  let confidence = 0.7;

  // 响应长度合理（太短可能不完整，太长可能有幻觉）

```

```

    if (response.content.length > 100 && response.content.length < 10000) {
        confidence += 0.1;
    }

    // 包含必需字段
    if (typeof data === 'object' && data !== null) {
        const keys = Object.keys(data);
        if (keys.length >= 3) {
            confidence += 0.1;
        }
    }

    return Math.min(confidence, 1);
}

/**
 * 解析变量值
 */
protected resolveVariable(name: string, context: AgentContext): unknown {
    switch (name) {
        case 'domains':
            return context.knowledgeGraph?.domains ?? [];
        case 'domainPatterns':
            return context.knowledgeGraph?.patterns ?? [];
        case 'technicalConstraints':
            return context.eab?.constraints ?? {};
        case 'eab':
            return context.eab ?? {};
        default:
            return undefined;
    }
}
}
}

```

F-9.4 需求分析师智能体

```

/**
 * 需求分析师智能体
 *
 * 核心能力：
 * 1. 理解模糊的业务需求
 * 2. 主动追问业务潜规则
 * 3. 提供策略选项（不是唯一方案）
 * 4. 生成领域模型和用户故事
 */

import { BaseAgent, type AgentContext, type AgentResponse } from '../base';
import { REQUIREMENT_ANALYST_PROMPT } from '../llm/prompts';
import type { LLMGateway } from '../llm/types';

export interface RequirementAnalysis {
    /** 对需求的理解 */
    understanding: string;
    /** 追问的问题 */
    questions: string[];
    /** 提议的领域模型 */
    proposedDomainModel: {
        entities: { name: string; fields: { name: string; type: string }[] }[];
    };
}

```

```

    relationships: { from: string; to: string; type: string }[];
    businessRules: { name: string; condition: string; action: string }[];
};
/** 策略选项 */
strategies: {
    name: string;
    description: string;
    pros: string[];
    cons: string[];
    impact: string;
}[];
/** 用户故事 */
userStories: {
    role: string;
    action: string;
    goal: string;
    acceptance: string;
}[];
/** AI 识别的隐含需求 */
implicitRequirements: string[];
/** 风险提示 */
risks: string[];
}

export class RequirementAnalystAgent extends BaseAgent {
    constructor(llm: LLMGateway) {
        super(llm, REQUIREMENT_ANALYST_PROMPT);
    }

    /**
     * 分析需求
     */
    async analyze(userInput: string, context: AgentContext):
    Promise<AgentResponse<RequirementAnalysis>> {
        return this.execute<RequirementAnalysis>(userInput, context);
    }

    /**
     * 追问（当用户回答了第一轮问题后，继续深入）
     */
    async followUp(
        userAnswers: Record<string, string>,
        previousAnalysis: RequirementAnalysis,
        context: AgentContext
    ): Promise<AgentResponse<RequirementAnalysis>> {
        const followUpInput = `
用户回答了之前的追问：
${Object.entries(userAnswers).map(([q, a]) => `问: ${q}\n答: ${a}`)}.join('\n\n')
`;

        return this.execute<RequirementAnalysis>(followUpInput, {
            ...context,

```

```

    messages: [
      ...context.messages,
      { role: 'assistant', content: JSON.stringify(previousAnalysis) },
    ],
  });
}

/**
 * 评估置信度（需求分析特化）
 */
protected evaluateConfidence(data: RequirementAnalysis): number {
  let confidence = 0.6;

  // 识别了实体
  if (data.proposedDomainModel?.entities?.length > 0) confidence += 0.1;
  // 识别了业务规则
  if (data.proposedDomainModel?.businessRules?.length > 0) confidence += 0.1;
  // 提供了策略选项
  if (data.strategies?.length >= 2) confidence += 0.1;
  // 生成了用户故事
  if (data.userStories?.length > 0) confidence += 0.1;

  return Math.min(confidence, 1);
}
}

```

F-9.5 架构师智能体

```

/**
 * 架构师智能体
 *
 * 核心能力：
 *   1. 将领域模型转化为技术架构
 *   2. 从 EAB 白名单中选择技术方案
 *   3. 生成 IR（表单、列表、API）
 *   4. 确保架构符合平台约束
 */

import { BaseAgent, type AgentContext, type AgentResponse } from '../base';
import { ARCHITECT_PROMPT } from '../llm/prompts';
import type { LLMGateway } from '../llm/types';
import type { FormPageIR } from '../..ir/types';

export interface ArchitectureDesign {
  /** 架构概述 */
  overview: string;
  /** 模块设计 */
  modules: {
    name: string;
    description: string;
    entities: string[];
    apis: { path: string; method: string; description: string }[][];
  }[][];
  /** 数据库设计 */
  database: {
    tables: {
      name: string;
      columns: { name: string; type: string; nullable: boolean; comment: string }[][];
    };
  };
}

```

```

        indexes: { name: string; columns: string[]; unique: boolean }[];
    }[];
};
/** 生成的 IR 列表 */
irPages: FormPageIR[];
/** 技术选型 */
techStack: {
    framework: string;
    ui: string;
    database: string;
    cache: string;
    mq: string;
};
/** 设计决策 */
decisions: {
    decision: string;
    reason: string;
    alternatives: string[];
}[];
}

```

```

export class ArchitectAgent extends BaseAgent {
    constructor(llm: LLMGateway) {
        super(llm, ARCHITECT_PROMPT);
    }

```

```

    /**

```

```

     * 设计架构

```

```

    */

```

```

    async design(
        requirementAnalysis: string,
        context: AgentContext
    ): Promise<AgentResponse<ArchitectureDesign>> {
        const input = `

```

基于以下需求分析，设计系统架构：

```

${requirementAnalysis}

```

请输出完整的架构设计，包括：

1. 模块划分
 2. 数据库设计（表结构、索引）
 3. API 设计
 4. IR（表单和列表的中间表示）
 5. 技术选型（从 EAB 白名单中选择）
 6. 设计决策和理由
- ```

`;

```

```

 return this.execute<ArchitectureDesign>(input, context);
 }

```

```

 /**

```

```

 * 优化架构（当用户提出修改意见后）

```

```

 */

```

```

 async optimize(
 feedback: string,
 currentDesign: ArchitectureDesign,
 context: AgentContext
): Promise<AgentResponse<ArchitectureDesign>> {

```

```
const input = `
用户对当前架构提出了以下修改意见：
${feedback}

当前架构设计：
${JSON.stringify(currentDesign, null, 2)}

请根据用户意见优化架构，保持整体一致性。
`;

return this.execute<ArchitectureDesign>(input, {
 ...context,
 currentIR: currentDesign as unknown as Record<string, unknown>,
});
}
}
```

---

## Week 5-6: UI/UX 设计智能体 + 数据库智能体

### F-9.6 UI/UX 设计智能体

```
/**
 * UI/UX 设计智能体
 *
 * 核心能力：
 * 1. 调用"设计DNA"文件，生成符合品牌规范的 UI
 * 2. 基于业务场景选择合适的布局模式
 * 3. 生成高保真页面 IR
 * 4. 自动填充 aiHints.designRationale
 */

import { BaseAgent, type AgentContext, type AgentResponse } from '../base';
import type { LLMGateway } from '../llm/types';
import type { FormPageIR, DashboardIR } from '../..ir/types';

export interface UIDesign {
 /** 设计概述 */
 overview: string;
 /** 页面类型 */
 pageType: 'form' | 'list' | 'dashboard' | 'detail';
 /** 设计理由 */
 designRationale: string;
 /** 布局方案 */
 layout: {
 type: 'grid' | 'flex' | 'absolute';
 columns?: number;
 gap?: number;
 responsive?: boolean;
 };
 /** 配色方案 */
 colorScheme: {
 primary: string;
 secondary: string;
 background: string;
 text: string;
 };
};
```



```

/** 生成的 IR */
ir: FormPageIR | DashboardIR;
/** 交互说明 */
interactions: {
 trigger: string;
 action: string;
 animation?: string;
}[];
}

export class UIUXAgent extends BaseAgent {
 constructor(llm: LLMGateway) {
 super(llm, {
 id: 'ui-ux-designer',
 name: 'UI/UX 设计师',
 systemPrompt: `你是一位资深的 UI/UX 设计师，精通 JNPF 低代码平台的组件体系。

```

设计 DNA（品牌规范）：  
 {{designDNA}}

可用组件：  
 {{availableComponents}}

你的职责：

1. 基于业务场景选择合适的页面布局
2. 遵循设计 DNA 保持视觉一致性
3. 生成符合 JNPF IR 格式的页面设计
4. 为每个设计决策提供理由

设计原则：

- 信息层次清晰（重要信息突出显示）
- 操作路径最短（常用操作减少点击）
- 响应式适配（PC + 移动端）
- 无障碍设计（颜色对比度、键盘导航）

输出 JSON 格式：

```

{
 "overview": "设计概述",
 "pageType": "form/list/dashboard/detail",
 "designRationale": "设计理由",
 "layout": { "type": "grid/flex/absolute", ... },
 "colorScheme": { ... },
 "ir": { ... },
 "interactions": [...]
}`,
 variables: [
 { name: 'designDNA', description: '品牌设计规范', required: true },
 { name: 'availableComponents', description: '可用组件列表', required: true },
],
});
}

async design(
 requirement: string,
 context: AgentContext
): Promise<AgentResponse<UIDesign>> {
 return this.execute<UIDesign>(requirement, context);
}

```

```
}
```

## F-9.7 数据库与后端智能体

```
/**
 * 数据库与后端智能体
 *
 * 核心能力：
 * 1. 生成数据模型 IR
 * 2. 注入多租户、审计等基础契约
 * 3. 生成 API 设计
 * 4. 生成数据库迁移脚本
 */

import { BaseAgent, type AgentContext, type AgentResponse } from './base';
import type { LLMGateway } from '../llm/types';

export interface DatabaseDesign {
 /** 数据库概述 */
 overview: string;
 /** 表设计 */
 tables: {
 name: string;
 comment: string;
 columns: {
 name: string;
 type: string;
 length?: number;
 nullable: boolean;
 defaultValue?: string;
 comment: string;
 /** 是否为审计字段（自动注入） */
 isAudit?: boolean;
 /** 是否为租户字段（自动注入） */
 isTenant?: boolean;
 }[];
 indexes: {
 name: string;
 columns: string[];
 unique: boolean;
 }[];
 }[];
 /** 迁移脚本（SQL） */
 migrationsSql: string;
 /** API 设计 */
 apis: {
 path: string;
 method: 'GET' | 'POST' | 'PUT' | 'DELETE';
 description: string;
 requestType?: string;
 responseType?: string;
 /** 是否需要权限校验 */
 requireAuth: boolean;
 /** 权限码 */
 permissionCode?: string;
 }[];
}
```

```
export class DatabaseAgent extends BaseAgent {
 constructor(llm: LLMGateway) {
 super(llm, {
 id: 'database-designer',
 name: '数据库与后端设计师',
 systemPrompt: `你是一位资深的数据库架构师，精通 JNPF 平台的数据层设计。`
 });
 }
}
```

JNPF 数据层约束：

- ORM: `SqlSugar` (Code First)
- 多租户：所有业务表必须包含 `TenantId` 列
- 审计字段： `CreateUserId`, `CreateTime`, `ModifyUserId`, `ModifyTime`
- 逻辑删除： `IsDeleted` 字段
- 主键策略：雪花算法 (`bigint`)
- 数据库： `SQL Server`

命名规范：

- 表名：大写字母 + 下划线（如 `SYS_USER`, `FLOW_TASK`）
- 模块前缀： `BASE_`（基础）， `EXT_`（扩展）， `FLOW_`（工作流）， `IOT_`（物联网）， `MES_`（制造）
- 字段名：大写字母 + 下划线

输出 JSON 格式：

```
{
 "overview": "数据库设计概述",
 "tables": [...],
 "migrationSql": "...",
 "apis": [...]
},
{
 variables: [],
 });
}
```

```
async design(
 domainModel: string,
 context: AgentContext
): Promise<AgentResponse<DatabaseDesign>> {
 // 自动注入多租户和审计字段
 const input = `
```

基于以下领域模型，设计数据库表结构和 API：

`${domainModel}`

请确保：

1. 所有业务表包含 `TenantId` (`NVARCHAR(50)`) 列
  2. 所有表包含审计字段 (`CreateUserId`, `CreateTime`, `ModifyUserId`, `ModifyTime`)
  3. 所有表包含逻辑删除字段 (`IsDeleted BIT DEFAULT 0`)
  4. 主键使用雪花算法 (`BIGINT`)
  5. 遵循 JNPF 命名规范
  6. 生成 `SQL Server` 迁移脚本
- ```
`;
```

```
    return this.execute<DatabaseDesign>(input, context);
  }
}
```

Week 7-8: 业务规则配置中心 + 无 AI 专家模式

F-9.8 业务规则配置中心

这是解决 AI 业务逻辑"死穴"的人类接口。

AI 生成的业务规则可能不准确，
人类专家在配置中心进行精修和裁决。

两种模式产出同构的 IR，保证 AI 降智时人类可接管。

```
/**
 * 业务规则引擎
 *
 * 支持三种规则类型：
 * 1. 决策表 (Decision Table) -- 多条件组合
 * 2. 决策树 (Decision Tree) -- 层级判断
 * 3. 规则链 (Rule Chain) -- 顺序执行
 */

export interface BusinessRule {
  id: string;
  name: string;
  description: string;
  type: 'decision-table' | 'decision-tree' | 'rule-chain';
  /** 关联的实体 */
  entity: string;
  /** 关联的字段 */
  fields: string[];
  /** 规则配置 */
  config: DecisionTable | DecisionTree | RuleChain;
  /** 来源: ai-generated / human-created / hybrid */
  source: 'ai-generated' | 'human-created' | 'hybrid';
  /** 版本 */
  version: number;
  /** 是否启用 */
  enabled: boolean;
}

/** 决策表 */
export interface DecisionTable {
  /** 条件列 */
  conditions: { field: string; operator: string; label: string }[];
  /** 动作列 */
  actions: { field: string; value: string; label: string }[];
  /** 规则行 */
  rows: {
    conditions: string[]; // 每个条件的值
    actions: string[]; // 每个动作的值
    priority: number;
  }[];
}

/** 决策树 */
export interface DecisionTree {
  nodes: {
    id: string;
```

```

    type: 'condition' | 'action';
    /** 条件节点：字段 + 操作符 + 值 */
    condition?: { field: string; operator: string; value: string };
    /** 动作节点：设置字段值 / 调用 API / 显示消息 */
    action?: { type: string; params: Record<string, unknown> };
    /** 条件为 true 时的下一个节点 */
    trueBranch?: string;
    /** 条件为 false 时的下一个节点 */
    falseBranch?: string;
  }[];
  rootNodeId: string;
}

/** 规则链 */
export interface RuleChain {
  rules: {
    id: string;
    condition: string; // 表达式
    action: string; // 表达式
    stopOnMatch: boolean; // 匹配后是否停止
  }[];
}

```

F-9.9 无 AI 专家模式 (逃生舱)

```

/**
 * 无 AI 专家模式
 *
 * 当 AI 降智或不可用时，无缝切换到手动模式。
 * 所有 AI 生成的功能退化为可视化手动操作。
 *
 * 关键：两种模式产出同构的 IR
 */

export interface ExpertModeConfig {
  /** 是否启用 AI 模式 */
  aiEnabled: boolean;
  /** 当前 AI 供应商状态 */
  aiStatus: 'healthy' | 'degraded' | 'offline';
  /** 降级原因 */
  degradeReason?: string;
}

/**
 * 检测 AI 状态，自动切换模式
 */
export async function detectAIBotMode(llm: LLMGateway): Promise<ExpertModeConfig> {
  try {
    const healthy = await llm.healthCheck();
    return {
      aiEnabled: healthy,
      aiStatus: healthy ? 'healthy' : 'degraded',
    };
  } catch {
    return {
      aiEnabled: false,
      aiStatus: 'offline',
      degradeReason: 'AI 服务不可达',
    };
  }
}

```

```

    };
}
}

/**
 * 专家模式工具集
 * 当 AI 不可用时，提供以下手动工具：
 *
 * 1. 领域模型画板（拖拽式实体关系设计）
 * 2. 架构图设计器（从 EAB 快照中选择组件）
 * 3. 决策表编辑器（可视化配置业务规则）
 * 4. 表单设计器（已有，不改变）
 * 5. 大屏设计器（已有，不改变）
 *
 * 所有工具产出的都是 IR，与 AI 生成的 IR 同构。
 */

```

Week 9-10: 集成测试 + DKEE V1.0

F-9.10 DKEE V1.0 (领域知识进化引擎)

```

/**
 * DKEE V1.0 - 领域知识进化引擎
 *
 * 核心功能：
 * 1. 记录人类在配置中心做出的规则修改
 * 2. 从修改中提炼领域模式
 * 3. 将模式沉淀到知识图谱
 * 4. 下次同类场景主动调用
 */

export interface DomainPattern {
  id: string;
  name: string;
  domain: string;
  description: string;
  /** 模式来源 */
  source: 'ai-discovered' | 'human-created' | 'self-play';
  /** 模式内容（IR 片段） */
  pattern: {
    entities: unknown[];
    rules: unknown[];
    components: unknown[];
  };
  /** 使用次数 */
  usageCount: number;
  /** 成功率（自博弈中通过测试的比例） */
  successRate: number;
  /** 版本 */
  version: number;
}

/**
 * 观察人类操作，提炼领域模式
 */
export function observeAndExtract(

```

```

humanActions: {
  type: 'create' | 'modify' | 'delete';
  target: string;
  before: unknown;
  after: unknown;
}[],
currentDomain: string
): DomainPattern | null {
  // 分析操作模式
  const createActions = humanActions.filter(a => a.type === 'create');
  const modifyActions = humanActions.filter(a => a.type === 'modify');

  // 如果人类创建了新的实体/规则，可能是新领域模式
  if (createActions.length >= 3) {
    const entities = createActions
      .filter(a => a.target.startsWith('entity'))
      .map(a => a.after);

    const rules = createActions
      .filter(a => a.target.startsWith('rule'))
      .map(a => a.after);

    if (entities.length > 0 || rules.length > 0) {
      return {
        id: `pattern-${Date.now()}`,
        name: `${currentDomain}-模式-${new Date().toISOString().slice(0, 10)}`,
        domain: currentDomain,
        description: `从人类操作中提炼的 ${currentDomain} 领域模式`,
        source: 'human-created',
        pattern: { entities, rules, components: [] },
        usageCount: 0,
        successRate: 0,
        version: 1,
      };
    }
  }

  return null;
}

```

阶段五交付物

- | | |
|---|---------------|
| □ src/core/ai/llm/types.ts | - 大模型网关接口 |
| □ src/core/ai/llm/deepseek.ts | - DeepSeek 实现 |
| □ src/core/ai/llm/fallback.ts | - 多供应商降级网关 |
| □ src/core/ai/llm/prompts.ts | - Prompt 模板管理 |
| □ src/core/ai/agents/base.ts | - 智能体基类 |
| □ src/core/ai/agents/requirement-analyst.ts | - 需求分析师 |
| □ src/core/ai/agents/architect.ts | - 架构师智能体 |
| □ src/core/ai/agents/ui-ux.ts | - UI/UX 设计师 |
| □ src/core/ai/agents/database.ts | - 数据库设计师 |
| □ src/core/ai/rules/engine.ts | - 业务规则引擎 |
| □ src/core/ai/rules/decision-table.ts | - 决策表 |
| □ src/core/ai/rules/decision-tree.ts | - 决策树 |
| □ src/core/ai/expert-mode.ts | - 无 AI 专家模式 |
| □ src/core/ai/dkee/v1.ts | - DKEE V1.0 |
| □ src/views/ai/workbench.vue | - AI 对话工作台 |
| □ src/views/ai/ruleEditor.vue | - 业务规则配置中心 |

阶段五里程碑验收

- 大模型网关支持 DeepSeek + 通义千问 + 本地模型
- 多供应商降级切换成功
- 需求分析师能理解模糊需求并追问
- 架构师能基于 EAB 生成架构设计
- UI/UX 能生成符合设计 DNA 的页面 IR
- 数据库设计师能生成带多租户/审计的表结构
- 业务规则配置中心可手动编辑决策表/决策树
- AI 降智时自动切换到专家模式
- DKEE V1.0 能从人类操作中提炼领域模式
- AI 对话工作台可完成"需求→架构→代码生成"全链路

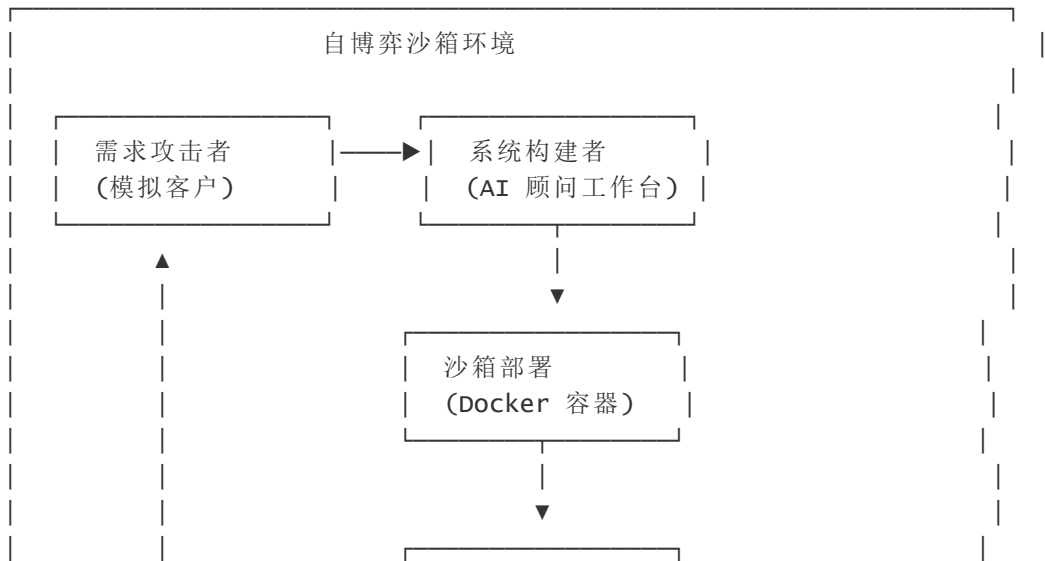
阶段六：多租户沙箱 + 创始人管理 + Foundry 对接 (8 周, Studio 侧)

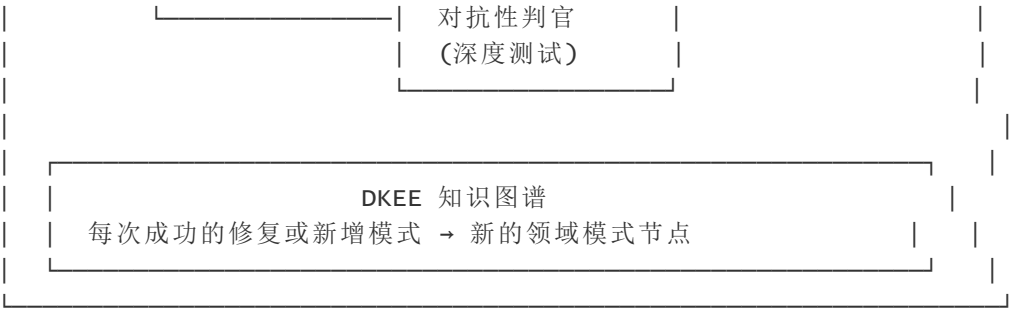
v5.0 边界：自博弈训练引擎 物理迁移至 Baobab-Foundry (独立 ~16 周, LLM Agentic Loop, 见第二篇 §4)。Studio 阶段六交付：**Docker 沙箱调度 (共享 SQL Server per-tenant DB)、FounderGuard + TOTP、KnowledgePatch 接收 → SQL Server BASE_KNOWLEDGE_*、创始人控制台 UI (转发 Foundry API, 不内嵌训练引擎)**。下文 F-10.x 为 Foundry 参考实现。

Studio 侧目标 (8 周)

- ① 混合多租户：逻辑隔离 (TenantId 全链路, MultiTenancy 启用) + 物理隔离 (Docker 沙箱, SemaphoreSlim 5 并发)
沙箱修正：所有沙箱 **共享一个 SQL Server 实例** (per-tenant database)；容器仅跑 API + 静态前端
- ② 创始人管理：FounderGuardMiddleware + TOTP + 创始人菜单 (模型与 Prompt 配置、自博弈开关 → Foundry API)
- ③ Foundry 对接：**HTTPS + 签名 zip** + KnowledgePatch 签名验证 + **SQL Server 知识表** 增量合并
- ④ 无 AI 专家模式：AI 降智时 VisualDev + IR 手工设计器，产出同构 IR
- ⑤ 流水线阶段 5 交付：测试 URL + 增量修改 + ZIP 导出

架构总览





Studio 侧交付清单（8 周，先于 Foundry 规格阅读）

周次	交付项	验收
W1-2	Docker 沙箱调度器 + TenantId 中间件 + 共享 SQL Server	5 并发; 30s 创建/销毁
W3-4	FounderGuard + TOTP + BASE_FOUNDER_AUTH_LOG	/api/founder 403/401 矩阵通过
W5-6	KnowledgePatch 签名验证 + BASE_KNOWLEDGE_* SQL 合并	1 次 Foundry→Studio 联调
W7	创始人控制台 UI + 图谱浏览器 (SQL 查询)	API 转发 Foundry, 不内嵌引擎
W8	无 AI 专家模式 + 流水线阶段 5 交付	VisualDev round-trip + ZIP 导出

以下 F-10.x 章节: Baobab-Foundry **独立 ~16 周**技术规格 (LLM Agentic Loop) ; Studio 不部署训练进程, 经 KnowledgePatch 接收经创始人签发的知识增量 (**SQL Server**; Foundry MVP2 再评估 Neo4j) 。

Foundry 参考规格 — Week 1-4: 沙箱环境 + 需求攻击者

F-10.1 沙箱环境管理器（Foundry 训练沙箱; Studio 客户沙箱见上表 W1-2)

```
/**
 * 沙箱环境管理器
 *
 * 为自博弈引擎提供隔离的测试环境:
 *   - 每个测试场景独立容器
 *   - 快速创建/销毁 (< 30 秒)
 *   - 包含完整 JNPF 单体 + 测试数据库
 *   - 资源限制 (防止失控)
 */

export interface SandboxConfig {
  /** 沙箱 ID */
  id: string;
  /** 用于哪个领域 */
  domain: string;
  /** 资源限制 */
  resources: {
    cpu: string;      // 如 '2'
    memory: string;   // 如 '4Gi'
    timeout: number;  // 最大存活时间 (ms)
  };
  /** 测试数据库配置 */
  database: {
    type: 'sqlite' | 'sqlserver';
    initScript?: string;
  };
};
```

```

}

export interface SandboxInstance {
  id: string;
  status: 'creating' | 'ready' | 'testing' | 'destroying' | 'destroyed';
  createdAt: number;
  /** 访问地址 */
  url: string;
  /** 数据库连接串 */
  dbConnectionString: string;
}

export class SandboxManager {
  private instances = new Map<string, SandboxInstance>();

  /**
   * 创建沙箱
   */
  async create(config: SandboxConfig): Promise<SandboxInstance> {
    const instance: SandboxInstance = {
      id: config.id,
      status: 'creating',
      createdAt: Date.now(),
      url: `http://sandbox-${config.id}:3100`,
      dbConnectionString: `Server=sandbox-db-${config.id};Database=JNPF_Test;`,
    };

    this.instances.set(config.id, instance);

    // 实际实现中，这里会调用 Docker API 创建容器
    // docker run -d --name sandbox-${config.id} ...

    // 模拟创建延迟
    await new Promise(r => setTimeout(r, 2000));

    instance.status = 'ready';
    return instance;
  }

  /**
   * 部署 IR 到沙箱
   */
  async deploy(sandboxId: string, project: GeneratedProject): Promise<void> {
    const instance = this.instances.get(sandboxId);
    if (!instance) throw new Error(`沙箱 ${sandboxId} 不存在`);

    instance.status = 'testing';

    // 实际实现中，这里会：
    // 1. 将 GeneratedProject 写入容器
    // 2. 执行 pnpm install && pnpm build
    // 3. 启动应用
    // 4. 初始化数据库
  }

  /**
   * 销毁沙箱
   */
}

```

```

async destroy(sandboxId: string): Promise<void> {
  const instance = this.instances.get(sandboxId);
  if (!instance) return;

  instance.status = 'destroying';

  // docker rm -f sandbox-${sandboxId}

  instance.status = 'destroyed';
  this.instances.delete(sandboxId);
}

/**
 * 批量销毁所有沙箱
 */
async destroyAll(): Promise<void> {
  for (const id of this.instances.keys()) {
    await this.destroy(id);
  }
}
}

```

F-10.2 需求攻击者智能体

```

/**
 * 需求攻击者智能体
 *
 * 使命：成为全世界最刁钻、最善变、最懂行的行业客户
 *
 * 能力：
 *   1. 从知识图谱中学习基础业务
 *   2. 创造性地生成各种需求场景
 *   3. 故意制造矛盾需求
 *   4. 生成极端边缘案例
 *   5. 模拟需求变更
 */

import type { LLMGateway } from '../llm/types';
import type { DomainPattern } from '../dkee/v1';

export interface AttackScenario {
  id: string;
  domain: string;
  /** 场景描述 */
  description: string;
  /** 需求列表 */
  requirements: {
    id: string;
    description: string;
    priority: 'must-have' | 'should-have' | 'nice-to-have';
    /** 是否为矛盾需求（故意制造） */
    isContradictory?: boolean;
    /** 矛盾的目标需求 ID */
    contradicts?: string;
  }[];
  /** 边缘案例 */
  edgeCases: {
    description: string;

```

```

    expectedResult: string;
  }[];
  /** 故障场景 */
  failureScenarios: {
    description: string;
    affectedComponents: string[];
  }[];
  /** 攻击难度 (1-10) */
  difficulty: number;
}

```

```

export class AttackerAgent {
  private llm: LLMGateway;
  private knowledgeBase: DomainPattern[];

  constructor(llm: LLMGateway, knowledgeBase: DomainPattern[]) {
    this.llm = llm;
    this.knowledgeBase = knowledgeBase;
  }

  /**
   * 生成攻击场景
   */
  async generateScenarios(
    domain: string,
    count: number,
    difficulty: number
  ): Promise<AttackScenario[]> {
    const relevantPatterns = this.knowledgeBase.filter(p => p.domain === domain);

    const prompt = `

```

你是一个极其刁钻的行业客户，正在测试一个 `${domain}` 领域的低代码平台。

已有的领域知识：

```
${JSON.stringify(relevantPatterns, null, 2)}
```

请生成 `${count}` 个攻击场景，难度级别 `${difficulty}/10`。

每个场景必须包含：

1. 真实的业务需求描述（像真正的行业客户一样说话）
2. 至少 3 个具体需求（其中 1 个故意与另一个矛盾）
3. 至少 2 个边缘案例（极端情况）
4. 至少 1 个故障场景（设备故障、网络中断等）

场景要覆盖：

- 基础功能（正常的业务流程）
- 并发冲突（多人同时操作同一资源）
- 状态异常（非法状态转换）
- 数据边界（超大/超小/空值/特殊字符）
- 权限越界（普通用户尝试管理员操作）

输出 JSON 数组格式：

```

[
  {
    "id": "scenario-1",
    "domain": "${domain}",
    "description": "...",
    "requirements": [...],

```

```

        "edgeCases": [...],
        "failureScenarios": [...],
        "difficulty": ${difficulty}
    }
}
];
`;

const response = await this.llm.chat({
    messages: [{ role: 'user', content: prompt }],
    responseFormat: 'json',
});

return JSON.parse(response.content) as AttackScenario[];
}

/**
 * 学习失败案例（判官反馈后更新攻击策略）
 */
learnFromFailure(scenario: AttackScenario, failure: string): void {
    // 将失败的攻击模式加入知识库
    // 下次生成场景时会参考这些模式
    console.log(`[Attacker] 学习到新的攻击模式: ${failure}`);
}
}

```

Week 5-8: 系统构建者自动化 + 对抗性判官

F-10.3 系统构建者自动化

```

/**
 * 系统构建者自动化
 *
 * 将阶段五的 AI 顾问工作台自动化运行：
 * 1. 接收攻击者的需求
 * 2. 自动调用四个智能体
 * 3. 自动生成 IR
 * 4. 自动编译为代码
 * 5. 自动部署到沙箱
 */

import type { AttackScenario } from '../attacker';
import type { RequirementAnalystAgent } from '../agents/requirement-analyst';
import type { ArchitectAgent } from '../agents/architect';
import type { DatabaseAgent } from '../agents/database';
import type { SandboxManager } from '../sandbox';
import { compileGateway } from '../../compiler/gateway';

export interface BuildResult {
    scenarioId: string;
    /** 是否成功构建 */
    success: boolean;
    /** 生成的 IR */
    ir?: unknown;
    /** 编译结果 */
    project?: Map<string, string>;
    /** 沙箱地址 */
}

```

```

sandboxUrl?: string;
/** 错误信息 */
error?: string;
/** 构建耗时 (ms) */
duration: number;
}

export class SystemBuilder {
  private requirementAgent: RequirementAnalystAgent;
  private architectAgent: ArchitectAgent;
  private databaseAgent: DatabaseAgent;
  private sandboxManager: SandboxManager;

  constructor(
    requirementAgent: RequirementAnalystAgent,
    architectAgent: ArchitectAgent,
    databaseAgent: DatabaseAgent,
    sandboxManager: SandboxManager
  ) {
    this.requirementAgent = requirementAgent;
    this.architectAgent = architectAgent;
    this.databaseAgent = databaseAgent;
    this.sandboxManager = sandboxManager;
  }

  /**
   * 从需求到部署的全自动流程
   */
  async buildAndDeploy(scenario: AttackScenario): Promise<BuildResult> {
    const start = Date.now();

    try {
      // Step 1: 需求分析
      const requirementResult = await this.requirementAgent.analyze(
        scenario.description + '\n\n具体需求: \n' + scenario.requirements.map(r =>
r.description).join('\n'),
        { messages: [], domain: scenario.domain }
      );

      // Step 2: 架构设计
      const architectureResult = await this.architectAgent.design(
        JSON.stringify(requirementResult.data),
        { messages: [], domain: scenario.domain }
      );

      // Step 3: 数据库设计
      const dbResult = await this.databaseAgent.design(
        JSON.stringify(requirementResult.data.proposedDomainModel),
        { messages: [], domain: scenario.domain }
      );

      // Step 4: 编译为代码
      const ir = architectureResult.data.irPages[0];
      if (!ir) throw new Error('架构师未生成 IR');

      const compileResult = await compileGateway({
        schema: ir,
        target: 'vue3-web',

```

```

        config: { entity: scenario.domain, entityLabel: scenario.description },
    });

    if (!compileResult.success || !compileResult.project) {
        throw new Error(`编译失败: ${compileResult.error}`);
    }

    // Step 5: 部署到沙箱
    const sandbox = await this.sandboxManager.create({
        id: `sandbox-${scenario.id}`,
        domain: scenario.domain,
        resources: { cpu: '2', memory: '4Gi', timeout: 300000 },
        database: { type: 'sqlite' },
    });

    await this.sandboxManager.deploy(sandbox.id, compileResult.project);

    return {
        scenarioId: scenario.id,
        success: true,
        ir,
        project: compileResult.project,
        sandboxUrl: sandbox.url,
        duration: Date.now() - start,
    };
} catch (e) {
    return {
        scenarioId: scenario.id,
        success: false,
        error: (e as Error).message,
        duration: Date.now() - start,
    };
}
}
}

```

F-10.4 对抗性判官智能体

```

/**
 * 对抗性判官智能体
 *
 * 使命：执行深度业务测试，判定系统是否正确
 *
 * 能力：
 * 1. 根据攻击者的意图生成测试用例
 * 2. 执行业务逻辑的语义验证（不是简单 HTTP 断言）
 * 3. 分析失败原因
 * 4. 生成修复建议
 */

import type { LLMGateway } from '../llm/types';
import type { AttackScenario } from './attacker';

export interface TestCase {
    id: string;
    scenarioId: string;
    description: string;
    type: 'functional' | 'boundary' | 'concurrent' | 'security' | 'performance';
}

```

```

/** 测试步骤 */
steps: {
  action: string;
  endpoint?: string;
  method?: string;
  body?: unknown;
  expectedResult: unknown;
}[];
/** 业务语义断言（由 AI 验证） */
semanticAssertions: {
  description: string;
  /** 如何验证（自然语言描述） */
  verification: string;
}[];
}

export interface TestResult {
  testCaseId: string;
  passed: boolean;
  /** 每一步的结果 */
  stepResults: {
    step: number;
    passed: boolean;
    actual: unknown;
    expected: unknown;
    error?: string;
  }[];
  /** 语义断言结果 */
  assertionResults: {
    assertion: string;
    passed: boolean;
    reasoning: string;
  }[];
  /** 失败分析 */
  failureAnalysis?: {
    rootCause: string;
    affectedComponents: string[];
    suggestedFix: string;
  };
}

export class JudgeAgent {
  private llm: LLMGateway;

  constructor(llm: LLMGateway) {
    this.llm = llm;
  }

  /**
   * 生成测试用例
   */
  async generateTestCases(scenario: AttackScenario): Promise<TestCase[]> {
    const prompt = `

```

你是一个极其严格的质量评审官，正在为以下场景生成测试用例：

场景描述：\${scenario.description}
 需求列表：\${JSON.stringify(scenario.requirements)}
 边缘案例：\${JSON.stringify(scenario.edgeCases)}

故障场景: `${JSON.stringify(scenario.failureScenarios)}`

请为每个需求和边缘案例生成详细的测试用例，包括：

1. 功能测试（正常流程）
2. 边界测试（极端值）
3. 并发测试（多用户同时操作）
4. 安全测试（权限越界）
5. 性能测试（大数据量）

每个测试用例必须包含：

- 明确的步骤
- 预期结果
- 至少 1 个业务语义断言（如"VIP 订单的完成时间必须早于普通订单"）

输出 JSON 数组格式。

```
`;  
  
const response = await this.llm.chat({  
  messages: [{ role: 'user', content: prompt }],  
  responseFormat: 'json',  
});  
  
return JSON.parse(response.content) as TestCase[];  
}  
  
/**  
 * 执行测试并分析结果  
 */  
async executeAndAnalyze(  
  testCase: TestCase,  
  sandboxUrl: string  
): Promise<TestResult> {  
  const stepResults: TestResult['stepResults'] = [];  
  
  // 执行每个测试步骤  
  for (let i = 0; i < testCase.steps.length; i++) {  
    const step = testCase.steps[i];  
    try {  
      const response = await fetch(`${sandboxUrl}${step.endpoint}`, {  
        method: step.method ?? 'GET',  
        headers: { 'Content-Type': 'application/json' },  
        body: step.body ? JSON.stringify(step.body) : undefined,  
      });  
  
      const actual = await response.json();  
      const passed = this.compareValues(actual, step.expectedResult);  
  
      stepResults.push({ step: i, passed, actual, expected: step.expectedResult });  
    } catch (e) {  
      stepResults.push({  
        step: i,  
        passed: false,  
        actual: null,  
        expected: step.expectedResult,  
        error: (e as Error).message,  
      });  
    }  
  }  
}
```

```

// AI 验证语义断言
const assertionResults = await this.verifySemanticAssertions(
    testCase.semanticAssertions,
    stepResults
);

const allPassed = stepResults.every(r => r.passed) && assertionResults.every(r =>
r.passed);

// 如果失败，生成失败分析
let failureAnalysis: TestResult['failureAnalysis'];
if (!allPassed) {
    failureAnalysis = await this.analyzeFailure(testCase, stepResults,
assertionResults);
}

return {
    testCaseId: testCase.id,
    passed: allPassed,
    stepResults,
    assertionResults,
    failureAnalysis,
};
}

```

```

/**
 * AI 验证语义断言
 */
private async verifySemanticAssertions(
    assertions: TestCase['semanticAssertions'],
    stepResults: TestResult['stepResults']
): Promise<TestResult['assertionResults']> {
    const prompt = `

```

基于以下测试执行结果，验证业务语义断言：

测试步骤结果：

```
${JSON.stringify(stepResults, null, 2)}
```

语义断言：

```
${JSON.stringify(assertions, null, 2)}
```

请逐条验证每个断言是否成立，并给出推理过程。

输出 JSON 格式：

```

[
  {
    "assertion": "断言描述",
    "passed": true/false,
    "reasoning": "推理过程"
  }
]
`
;

```

```

const response = await this.llm.chat({
    messages: [{ role: 'user', content: prompt }],
    responseFormat: 'json',
});

```

```

    return JSON.parse(response.content);
}

/**
 * 分析失败原因
 */
private async analyzeFailure(
    testCase: TestCase,
    stepResults: TestResult['stepResults'],
    assertionResults: TestResult['assertionResults']
): Promise<TestResult['failureAnalysis']> {
    const prompt = `
测试失败，请分析根本原因并给出修复建议：

测试用例：${JSON.stringify(testCase)}
步骤结果：${JSON.stringify(stepResults)}
断言结果：${JSON.stringify(assertionResults)}

请输出：
{
  "rootCause": "根本原因分析",
  "affectedComponents": ["受影响的组件1", "组件2"],
  "suggestedFix": "修复建议"
}
`;

    const response = await this.llm.chat({
        messages: [{ role: 'user', content: prompt }],
        responseFormat: 'json',
    });

    return JSON.parse(response.content);
}

private compareValues(actual: unknown, expected: unknown): boolean {
    return JSON.stringify(actual) === JSON.stringify(expected);
}
}

```

Week 9-12: 自动进化闭环

F-10.5 自博弈引擎

```

/**
 * 自博弈引擎
 *
 * 将三个智能体串联为自动化的闭环：
 * 1. 攻击者生成场景
 * 2. 构建者生成系统并部署
 * 3. 判官执行测试
 * 4. 失败 → 修复 → 重新测试
 * 5. 成功 → 沉淀到知识图谱
 * 6. 循环
 */

```

```

import type { AttackerAgent, AttackScenario } from './attacker';
import type { SystemBuilder, BuildResult } from './builder';
import type { JudgeAgent, TestResult } from './judge';
import type { SandboxManager } from './sandbox';

export interface SelfPlayConfig {
  /** 目标领域 */
  domain: string;
  /** 总循环次数 */
  totalRounds: number;
  /** 每轮生成的场景数 */
  scenariosPerRound: number;
  /** 攻击难度（会逐步提升） */
  initialDifficulty: number;
  /** 最大难度 */
  maxDifficulty: number;
  /** 每轮超时时间（ms） */
  roundTimeout: number;
}

export interface RoundResult {
  round: number;
  difficulty: number;
  scenarios: AttackScenario[];
  buildResults: BuildResult[];
  testResults: TestResult[];
  /** 通过率 */
  passRate: number;
  /** 发现的缺陷 */
  bugsFound: { scenarioId: string; description: string; fix: string }[];
  /** 沉淀的新模式 */
  newPatterns: unknown[];
  duration: number;
}

export interface SelfPlayReport {
  config: SelfPlayConfig;
  rounds: RoundResult[];
  /** 总体统计 */
  stats: {
    totalScenarios: number;
    totalPassed: number;
    totalFailed: number;
    overallPassRate: number;
    bugsFoundAndFixed: number;
    newPatternsLearned: number;
  };
  /** 知识图谱增长 */
  knowledgeGrowth: {
    before: number;
    after: number;
    newNodes: number;
  };
}

export class SelfPlayEngine {
  private attacker: AttackerAgent;
  private builder: SystemBuilder;

```

```

private judge: JudgeAgent;
private sandboxManager: SandboxManager;

constructor(
  attacker: AttackerAgent,
  builder: SystemBuilder,
  judge: JudgeAgent,
  sandboxManager: SandboxManager
) {
  this.attacker = attacker;
  this.builder = builder;
  this.judge = judge;
  this.sandboxManager = sandboxManager;
}

/**
 * 运行自博弈
 */
async run(config: SelfPlayConfig): Promise<SelfPlayReport> {
  const report: SelfPlayReport = {
    config,
    rounds: [],
    stats: { totalScenarios: 0, totalPassed: 0, totalFailed: 0, overallPassRate: 0,
    bugsFoundAndFixed: 0, newPatternsLearned: 0 },
    knowledgeGrowth: { before: 0, after: 0, newNodes: 0 },
  };

  let difficulty = config.initialDifficulty;

  for (let round = 1; round <= config.totalRounds; round++) {
    console.log(`[SelfPlay] === 第 ${round} 轮 (难度 ${difficulty}) ===`);

    const roundResult = await this.executeRound(config, round, difficulty);
    report.rounds.push(roundResult);

    // 更新统计
    report.stats.totalScenarios += roundResult.scenarios.length;
    report.stats.totalPassed += roundResult.testResults.filter(r =>
r.passed).length;
    report.stats.totalFailed += roundResult.testResults.filter(r =>
!r.passed).length;
    report.stats.bugsFoundAndFixed += roundResult.bugsFound.length;
    report.stats.newPatternsLearned += roundResult.newPatterns.length;

    // 逐步提升难度
    if (roundResult.passRate > 0.8 && difficulty < config.maxDifficulty) {
      difficulty++;
      console.log(`[SelfPlay] 通过率 ${roundResult.passRate * 100}%, 提升难度到
${difficulty}`);
    }

    // 清理本轮沙箱
    await this.sandboxManager.destroyAll();
  }

  report.stats.overallPassRate = report.stats.totalPassed /
report.stats.totalScenarios;

```

```

    return report;
}

/**
 * 执行一轮自博弈
 */
private async executeRound(
    config: SelfPlayConfig,
    round: number,
    difficulty: number
): Promise<RoundResult> {
    const start = Date.now();
    const bugsFound: RoundResult['bugsFound'] = [];
    const newPatterns: RoundResult['newPatterns'] = [];

    // Step 1: 攻击者生成场景
    const scenarios = await this.attacker.generateScenarios(
        config.domain,
        config.scenariosPerRound,
        difficulty
    );

    // Step 2: 对每个场景执行 构建→测试→修复 循环
    const buildResults: BuildResult[] = [];
    const testResults: TestResult[] = [];

    for (const scenario of scenarios) {
        // 构建
        const buildResult = await this.builder.buildAndDeploy(scenario);
        buildResults.push(buildResult);

        if (!buildResult.success) {
            testResults.push({
                testCaseId: scenario.id,
                passed: false,
                stepResults: [],
                assertionResults: [],
                failureAnalysis: { rootCause: buildResult.error!, affectedComponents: [],
suggestedFix: '检查编译错误' },
            });
            continue;
        }

        // 生成测试用例
        const testCases = await this.judge.generateTestCases(scenario);

        // 执行测试
        for (const testCase of testCases) {
            const testResult = await this.judge.executeAndAnalyze(testCase,
buildResult.sandboxUrl!);
            testResults.push(testResult);

            // 如果失败，尝试修复
            if (!testResult.passed && testResult.failureAnalysis) {
                bugsFound.push({
                    scenarioId: scenario.id,
                    description: testResult.failureAnalysis.rootCause,
                    fix: testResult.failureAnalysis.suggestedFix,

```

```

    });

    // 攻击者学习失败模式
    this.attacker.learnFromFailure(scenario,
testResult.failureAnalysis.rootCause);
    }
  }
}

const passRate = testResults.filter(r => r.passed).length / testResults.length;

return {
  round,
  difficulty,
  scenarios,
  buildResults,
  testResults,
  passRate,
  bugsFound,
  newPatterns,
  duration: Date.now() - start,
};
}
}

```

Week 13-16: 深度生成式测试引擎

F-10.6 深度业务测试生成器

```

/**
 * 深度业务测试生成器
 *
 * 不是简单的 API 测试，而是深入业务逻辑的语义验证
 *
 * 示例：
 *   "在 1000 次插单测试中，VIP 订单的完成时间必须始终早于普通订单"
 *   "设备故障后，所有受影响工序的状态必须自动变为等待重排"
 *   "当同时发生安全帽未佩戴报警和火灾报警时，必须以火灾优先"
 */

export interface DeepTestSuite {
  id: string;
  domain: string;
  name: string;
  description: string;
  /** 测试用例 */
  testCases: DeepTestCase[];
  /** 全局前置条件 */
  preconditions: string[];
  /** 全局清理操作 */
  cleanup: string[];
}

export interface DeepTestCase {
  id: string;
  name: string;

```

```

description: string;
type: 'stress' | 'chaos' | 'semantic' | 'regression';
/** 测试数据生成策略 */
dataStrategy: {
  /** 生成多少条数据 */
  count: number;
  /** 数据分布（如 "80% 普通订单，20% VIP 订单"） */
  distribution: string;
  /** 时间范围 */
  timeRange?: { start: string; end: string };
};
/** 故障注入 */
faultInjection?: {
  type: 'service-down' | 'network-delay' | 'database-lock' | 'resource-exhaustion';
  target: string;
  duration: number;
};
/** 业务语义断言 */
assertions: {
  description: string;
  /** 验证方式：SQL 查询 + 条件判断 */
  verification: string;
  /** 期望结果 */
  expected: string;
  /** 严重级别 */
  severity: 'critical' | 'major' | 'minor';
}[];
/** 执行次数（压力测试用） */
iterations?: number;
}

```

```
/**
```

```
 * 生成深度测试套件
```

```
*/
```

```
export async function generateDeepTestSuite(
```

```
  llm: LLMGateway,
```

```
  domain: string,
```

```
  domainKnowledge: string,
```

```
  systemIR: unknown
```

```
): Promise<DeepTestSuite> {
```

```
  const prompt = `
```

```
你是一个极其严格的质量工程师，正在为 ${domain} 领域的系统设计深度测试。
```

```
领域知识：
```

```
${domainKnowledge}
```

```
系统设计（IR）：
```

```
${JSON.stringify(systemIR, null, 2)}
```

```
请生成一套深度测试，包括：
```

1. 压力测试（1000+ 并发请求）
2. 混沌测试（随机故障注入）
3. 语义测试（业务逻辑正确性验证）
4. 回归测试（历史缺陷不再复现）

```
每个测试必须包含：
```

- 明确的业务语义断言（不是简单的 HTTP 状态码检查）

- 测试数据生成策略（多少数据、什么分布）
- 故障注入策略（如适用）

输出 JSON 格式：

```
{
  "id": "deep-test-${domain}",
  "domain": "${domain}",
  "name": "${domain} 深度测试套件",
  "description": "...",
  "testCases": [...],
  "preconditions": [...],
  "cleanup": [...]
}
`;

const response = await llm.chat({
  messages: [{ role: 'user', content: prompt }],
  responseFormat: 'json',
});

return JSON.parse(response.content) as DeepTestSuite;
}
```

Week 17-20: 知识图谱自动增长 + 首个领域训练

F-10.7 知识图谱自动增长

```
/**
 * 知识图谱自动增长
 *
 * 每次自博弈成功修复或新增模式，
 * 都被提炼为新的领域模式节点，
 * 带上通过测试的证明。
 */

export interface KnowledgeNode {
  id: string;
  type: 'entity' | 'rule' | 'pattern' | 'anti-pattern';
  domain: string;
  name: string;
  description: string;
  /** 节点内容 */
  content: unknown;
  /** 来源 */
  source: 'self-play' | 'human-created' | 'ai-discovered';
  /** 验证证明（通过了哪些测试） */
  proof: {
    testSuiteId: string;
    testCaseId: string;
    passedAt: string;
  }[];
  /** 使用统计 */
  usage: {
    totalUsed: number;
    successCount: number;
    failureCount: number;
  };
}
```

```

};
/** 版本历史 */
versions: {
  version: number;
  content: unknown;
  createdAt: string;
  reason: string;
}[];
}

/**
 * 从自博弈结果中提炼知识
 */
export function extractKnowledge(
  roundResult: RoundResult,
  domain: string
): KnowledgeNode[] {
  const nodes: KnowledgeNode[] = [];

  // 从成功的测试中提炼模式
  for (const testResult of roundResult.testResults) {
    if (testResult.passed) {
      nodes.push({
        id: `knowledge-${Date.now()}-${Math.random().toString(36).slice(2, 6)}`,
        type: 'pattern',
        domain,
        name: `验证通过的模式`,
        description: `在第 ${roundResult.round} 轮自博弈中通过测试`,
        content: testResult,
        source: 'self-play',
        proof: [{
          testSuiteId: `round-${roundResult.round}`,
          testCaseId: testResult.testCaseId,
          passedAt: new Date().toISOString(),
        }],
        usage: { totalUsed: 1, successCount: 1, failureCount: 0 },
        versions: [{
          version: 1,
          content: testResult,
          createdAt: new Date().toISOString(),
          reason: '自博弈中首次验证通过',
        }],
      });
    }
  }

  // 从修复的缺陷中提炼反模式
  for (const bug of roundResult.bugsFound) {
    nodes.push({
      id: `anti-pattern-${Date.now()}-${Math.random().toString(36).slice(2, 6)}`,
      type: 'anti-pattern',
      domain,
      name: `已知缺陷模式`,
      description: bug.description,
      content: { bug, fix: bug.fix },
      source: 'self-play',
      proof: [],
      usage: { totalUsed: 0, successCount: 0, failureCount: 0 },
    });
  }
}

```

```

    versions: [{
      version: 1,
      content: { bug, fix: bug.fix },
      createdAt: new Date().toISOString(),
      reason: '自博弈中发现的缺陷',
    }],
  });
}

return nodes;
}

```

F-10.8 首个领域训练——智能更衣柜

选择"智能更衣柜"作为首个训练领域：

逻辑相对封闭

核心场景清晰（借还、异常、多人共享、支付）

适合验证自博弈引擎的可行性

训练计划：

Round 1-5: 难度 1-3, 基础功能（借还、查询、统计）

Round 6-10: 难度 4-6, 异常场景（柜门故障、网络中断、并发冲突）

Round 11-15: 难度 7-8, 复杂业务（VIP 优先、支付对接、多终端同步）

Round 16-20: 难度 9-10, 极端场景（1000 用户同时借还、设备批量故障）

目标：

完成至少 10000 次自博弈循环

形成包含 100+ 领域模式的知识图谱

通过率从初始的 60% 提升到 95%+

阶段六交付物

- src/core/ai/sandbox/sandbox-manager.ts – 沙箱环境管理器
- src/core/ai/selfplay/attacker.ts – 需求攻击者智能体
- src/core/ai/selfplay/builder.ts – 系统构建者自动化
- src/core/ai/selfplay/judge.ts – 对抗性判官智能体
- src/core/ai/selfplay/engine.ts – 自博弈引擎
- src/core/ai/selfplay/deep-test.ts – 深度业务测试生成器
- src/core/ai/dkee/knowledge-graph.ts – 知识图谱自动增长
- docs/domains/smart-locker/ – 智能更衣柜领域训练结果
- 标签: v5.2-self-play-m1

阶段六里程碑验收（Studio 侧，8 周）

- Docker 沙箱调度器：5 并发稳定，单沙箱 1 核/1GB，30 秒内创建/销毁
- 多租户隔离：TenantId 全链路 + 越权访问测试通过
- FounderGuard + TOTP: /api/founder 非创始人 403；无 founder_token 401
- KnowledgePatch: 签名验证 + **BASE_KNOWLEDGE_* SQL 增量合并** + 版本管理
- 创始人控制台 UI: 自博弈开关/模型与 Prompt 配置/图谱审核/审计日志（API 转发 Foundry）
- 无 AI 专家模式: 降智切换 + IR 手工设计器 + VisualDev 逃生舱 round-trip
- 五阶段流水线阶段 5: 测试 URL + 增量修改 + ZIP 导出
- Foundry 联调: **HTTPS + 签名 zip** + 至少 1 次 KnowledgePatch 端到端接收（Foundry ~16 周计划）

Foundry 侧里程碑（独立项目，非 Studio 阶段六验收）

- 需求攻击者能生成 10+ 场景/轮
- 系统构建者全自动完成 需求→架构→编译→部署
- 对抗性判官能执行业务语义验证
- 自博弈引擎可 7×24 自动运行
- 深度测试覆盖压力/混沌/语义/回归四类
- 智能更衣柜领域完成 10000+ 循环；通过率 60%→95%+
- 知识图谱 100+ 经核验领域模式；经创始人签发 KnowledgePatch 推送 Studio

六个阶段完整交付物总览（v5.0）

前置（强制门禁）：

- Sprint 0-A 闭合 Sprint（10 项 P0 门禁）
- Sprint 0-B AI 基础设施地桩（10 项 + 8 补充门禁）
- PoC 门禁 Three.js（阶段二前）；uni-app X **暂缓**

阶段零（已完成）：

- ✅ F-0 ~ F-4 + F-5 + ADR-016 + src/core 83 vitest

阶段一（4 周）：

- F-5 收官 + F-6a 大屏编译器基础 + 后端 Sprint 1-3

阶段二（4 周，全量）：

- F-6b 完整 3D 数字孪生 VIP（事件绑定 DSL 并入表达式引擎）

阶段三（4 周）：

- F-7 UniApp **单轨**（标准 uni-app）+ FlowIR v1 + 后端清零收尾

阶段四（3 周）：

- F-8 统一编译网关 + ZIP 下载 + **ir-to-schema 官方回写**

阶段五（10 周，硬门禁）：

- 五阶段 AI 流水线 + Evals + 组件覆盖 ≥90% + 多角色 web UI

阶段六（8 周，Studio 侧）：

- 多租户沙箱（共享 SQL）+ FounderGuard + Foundry 对接 + KnowledgePatch → SQL

Baobab-Foundry（独立 **~16 周**，LLM Agentic Loop，并行）：

- 四 Agent + 蒸馏师 + 因果回放池（SQL JSON）→ 签发 Patch

主体项目 Studio 工期：Sprint 0-A/B + PoC + 阶段一~六 ≈ **49 周**

v2.0「46→28 周压缩」方案 **废止**（见附录 A、D）

以上是第一篇（F-0~F-10 工程施工包）完整内容。以下第二篇为 D 爷 7/8/9 三稿升格后的「自博弈 AI 低代码」确定版，与第一篇阶段编号映射，形成全平台重构唯一执行纲领。

#